MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

SELETIVA AULA 6

DP com bitmasks

joca
palao
vlacerda

rev. 1.1

CC BY NC SA

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Bitmasks

Motivação natural de bitmasks: Subsets

Gerar subconjuntos de forma curta e rápida

Quais são os subconjuntos de {A,B,C} ?

{}	{000}	0	1 = Elemento incluso
{C}	{001}	1	0 = Não incluso
{B}	{010}	2	
{A}	{011}	3	
{A}	{100}	4	
{A,C}	{101}	5	
{A,B}	{110}	6	
{A,B,C}	{111}	7	

Bit Shifting:

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

(1 << n) == 2^n

```
10000     (1 << 4) o 1 foi empurrado 4 vezes pra esquerda
-----                para esquerda: multiplica por 2
43210                para direita:  divide por 2
```

```
bitset<5>bt(16);
cout << bt;
// 10000
```

Cuidados:
-1ll : usar “ll” para long long (64 bits)
-Overflow

Operações básicas:

tricks.cpp

```
//Ligar i ésimo bit:
        (n |= (1 << i))
//Testar se o i ésimo bit está ligado:
        (n & (1 << i))
//Flipar o i ésimo bit:
        (n ^= (1 << i))
```

CodeImage

Tricks:

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

tricks.cpp

```cpp
// Testar se é potência de 2:
      !(n & (n-1))

// Contar quantos bits estão ligados:
      __builtin_popcount(n)
      __builtin_popcountll(n)

// Bitmask cheio de 1:
        ((1 << n) - 1)

// LSB:
        (n & -n)
```

CodeImage

tricks.cpp

```cpp
// gerar todos os subsets:

for(int mask = 0;mask < (1 << n);mask++){

  for(int idx = 0;idx < n;idx++){
    if(mask & (1 << idx)){
      //está presente no subset
      // lembrar dos parênteses,
      // o operador '==' tem preferência
    }
  }

}
```

CodeImage

Generalização:

Qualquer número é uma soma de potências da sua base.

Portanto , se um problema te pede para trabalhar com números de bases diferentes, você consegue gerar a representação dessa base tirando o mod e dividindo pela base até zerar.

1121 (3): 1 * 3^3 + 1 * 3^2 + 2 * 3^1 + 1 * 3^0

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

DP em subconjuntos

Assignment Problem

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Berland shop sells n kinds of juices. Each juice has its price ci. Each juice includes some set of vitamins in it. There are three types of vitamins: **vitamin "A", vitamin "B" and vitamin "C".** Each juice can contain **one, two or all three types** of vitamins in it.

Petya knows that he needs all three types of vitamins to stay healthy. **What is the minimum total price of juices that Petya has to buy to obtain all three vitamins?** Petya obtains some vitamin if he buys at least one juice containing it and drinks it.

n <= 1e6.

Exemplos

5
10 A
9 BC
11 CA
4 A
5 B

5
10 ABC
3 BC
2 AC
1 C
6 AB

Exemplos

5

10 A

9 BC

11 CA

4 A

5 B

Resposta: 13

5

10 ABC

3 BC

2 AC

1 C

6 AB

Resposta: 5

Explicação

Fazemos uma DP de 8 estados (2^3), onde **cada bit ligado do número representa uma vitamina consumida**.

Assim a DP de cada número representa o menor valor necessário para se chegar naquele estado.

Caso não seja possível chegar no estado final (7), printamos -1.

Bit 1 ligado -> possui a vitamina A

Bit 2 ligado -> possui a vitamina B

Bit 3 ligado -> possui a vitamina C

Resposta = **Menor valor possível para o estado 111 = DP[7].**

Como seria a transição?

Transição

Vamos supor que recebemos:
5 AC

- Criamos um mask com os bits 1 e 3 ligados.
- Para cada um dos subconjuntos “j” possíveis, ligamos esses dois bits.
- Checamos cada subconjunto j após adicionar os itens do mask, o custo melhora?

Transition.cpp

```
for (int j=0; j<(1<<BITS); j++) {
    int x = j | mask;
    dp[x] = min(dp[x], dp[j]+cost);
}
```

Código

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Vitamins.cpp

```cpp
const ll INF = 1e18; //número arbitrariamente grande

void solve(){
    vector<ll> dp(1<<BITS,INF)
    dp[0] = 0; //não é necessário nenhum custo para obter o conjunto vazio
    for (int i=0; i<n; i++) {
        int mask = 0;
        for (char vitamin: juice) {
            if (vitamin == 'A') mask |= 1; //liga o primeiro bit
            else if (vitamin == 'B')  mask |= 2; //liga o segundo bit
            else if (vitamin == 'C') mask |= 4; //liga o terceiro bit
        }

        for (int j=0; j<(1<<BITS); j++) {
            int x = j | mask; //para cada possível estado, liga os bits do mask.
            dp[x] = min(dp[x], dp[j]+cost);
        }
    }

    if (dp[7] == INF) cout << -1 << endl;
    else cout << dp[7] << endl;
}
```

Complexidade: O(2^BITS * N)

Generalização

- Utilizada quando o estado depende de um subconjunto dos elementos
- Complexidade típica: O(2^N * N)
- Útil para N <= 22

Aplicações:

- Qual a melhor forma de escolher determinado subconjunto?
- Qual o custo mínimo de visitar/tomar exatamente esses elementos?
- Qual o emparelhamento ótimo entre posições e tarefas?

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

DP em permutações

Assignment Problem

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

E. Food Allocation I

time limit per test: 3 seconds
memory limit per test: 256 megabytes

After a local forest tour gone wrong, you and n other survivors have been stranded in the wilderness. Due to your incredible leadership skills, you have taken command of the rest of the survivors and need to organize them in order to get the necessary supplies to survive.

There are n different types of food that each of the n survivors can collect. Each survivor can collect any of the types of food, but they each have different skill levels at collecting each type of food, which represent the amount of that food that they can collect.

You must assign each survivor to a single food type and ensure that your camp has a survivor assigned to each food type to maximize the value of the food you get. In this case, you and the survivors all pool together the collected food, so the value of the assignment is the **sum** of all the food collected by each survivor. How do you maximize this assignment of survivors?

Input

The first line will consist of a single integer n ($2 \le n \le 10$) giving the number of survivors. The next n lines will each consist of n integers where the jth integer on the ith line, s_{ij} ($0 \le s_{ij} \le 10^5$), gives the skill point value for survivor i when it is assigned food type j.

Output

Output a single integer giving the total value of the optimal assignment of survivors to tasks.

Example

input Copy

```
2
3 4
0 2
```

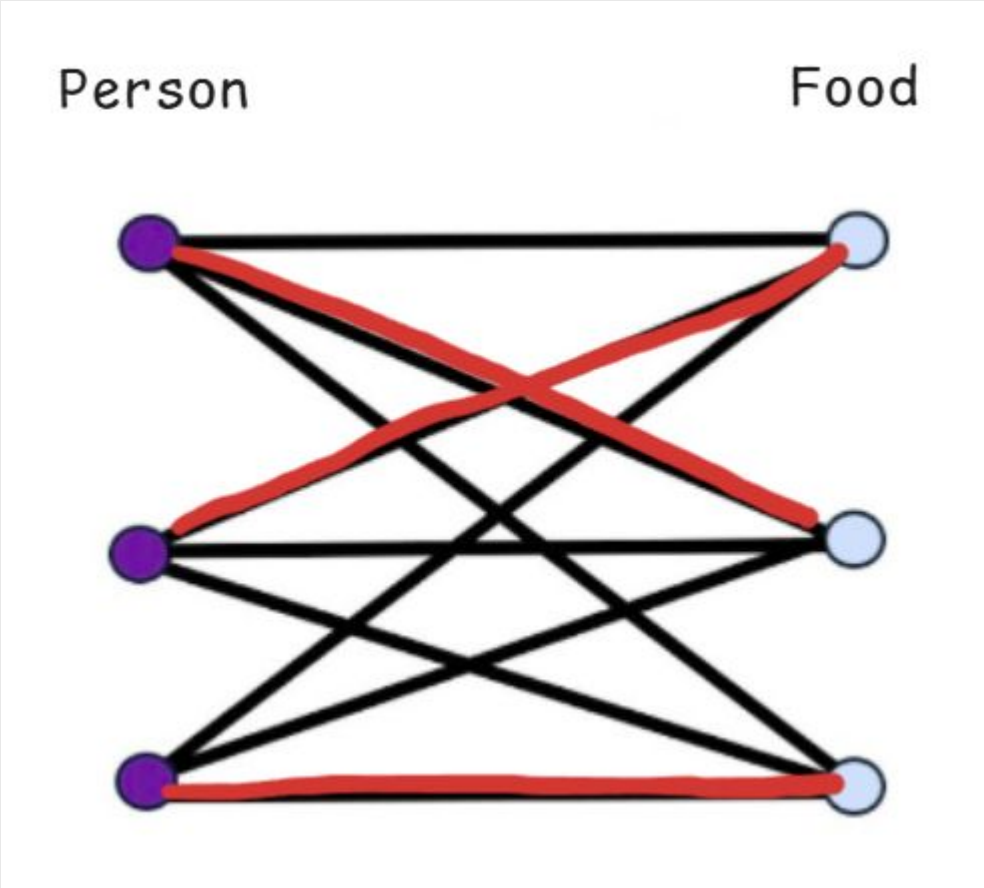
output Copy

```
5
```

Assignment

Uma atribuição vai ser um emparelhamento máximo num grafo bipartido.

O grafo bipartido é completo e os dois lados possuem o mesmo número de vértices ($K_{n,n}$)

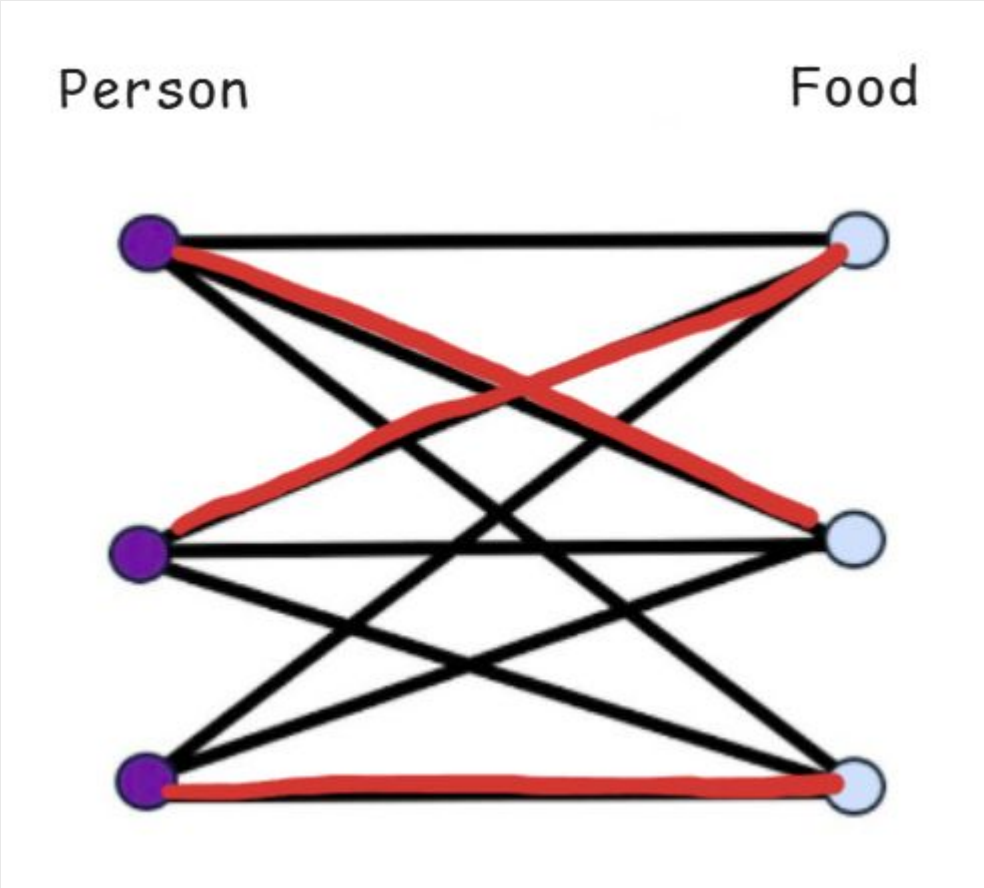


Assignment

Podemos associar cada assignment como uma permutação.

P[i] = comida escolhida pela pessoa i.

Podemos resolver em O(n!) testando todas as permutações

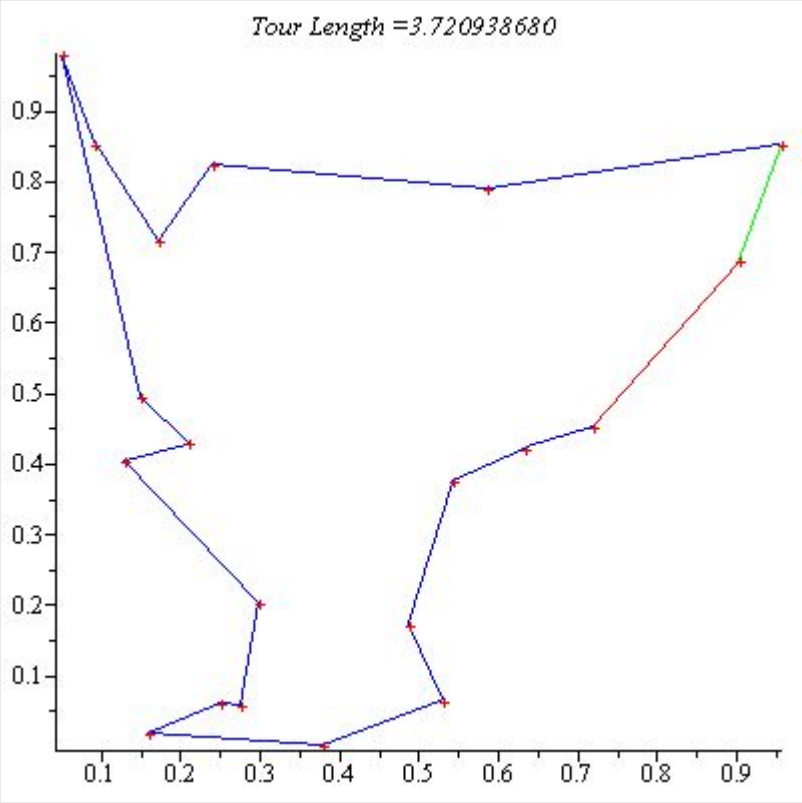


Assignment : [2,1,3]

Traveling Salesman Problem

Temos N pontos no 2D e queremos um ciclo hamiltoniano que minimize o custo total.

Em outras palavras, queremos achar uma **sequência** de visitações que minimize o custo total.



Traveling Salesman Problem

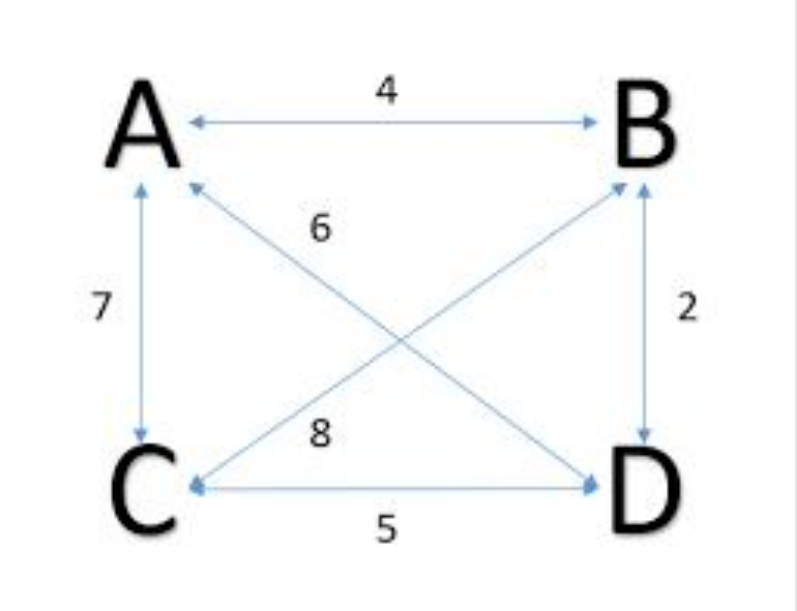
Podemos codificar cada caminho como uma permutação.

A->C->B->D: 7+8+2 = 17

A->B->D->C: 4+2+5 = 11

Cost(p) = Sum(C(p[i], p[i+1]))

Podemos resolver em O(n!) testando todas as permutações



Dependendo do problema, pode ser um caminho ou um ciclo

Dynamic Programming

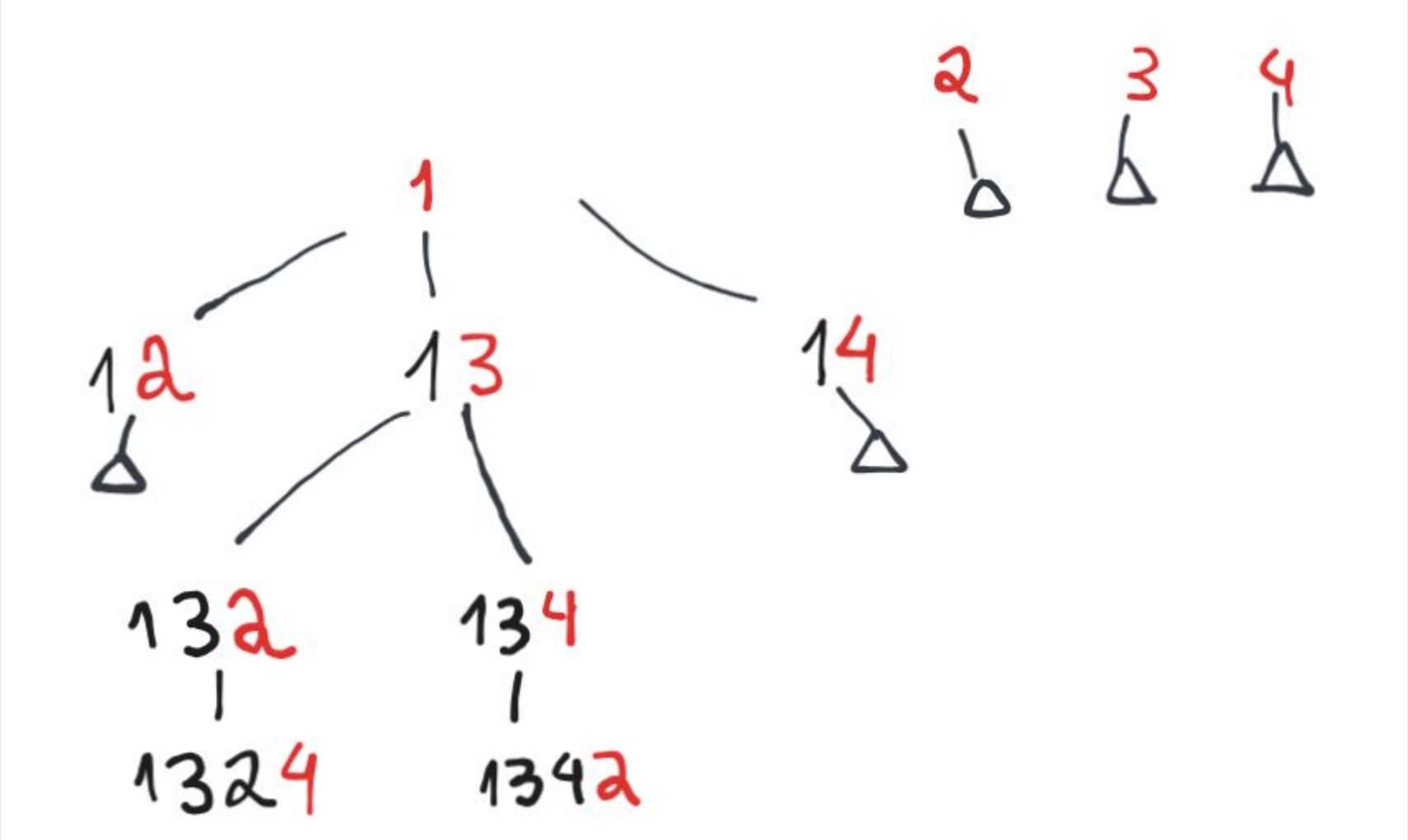
- Permutações

MaratonaCIn

Centro de Informática UFPE

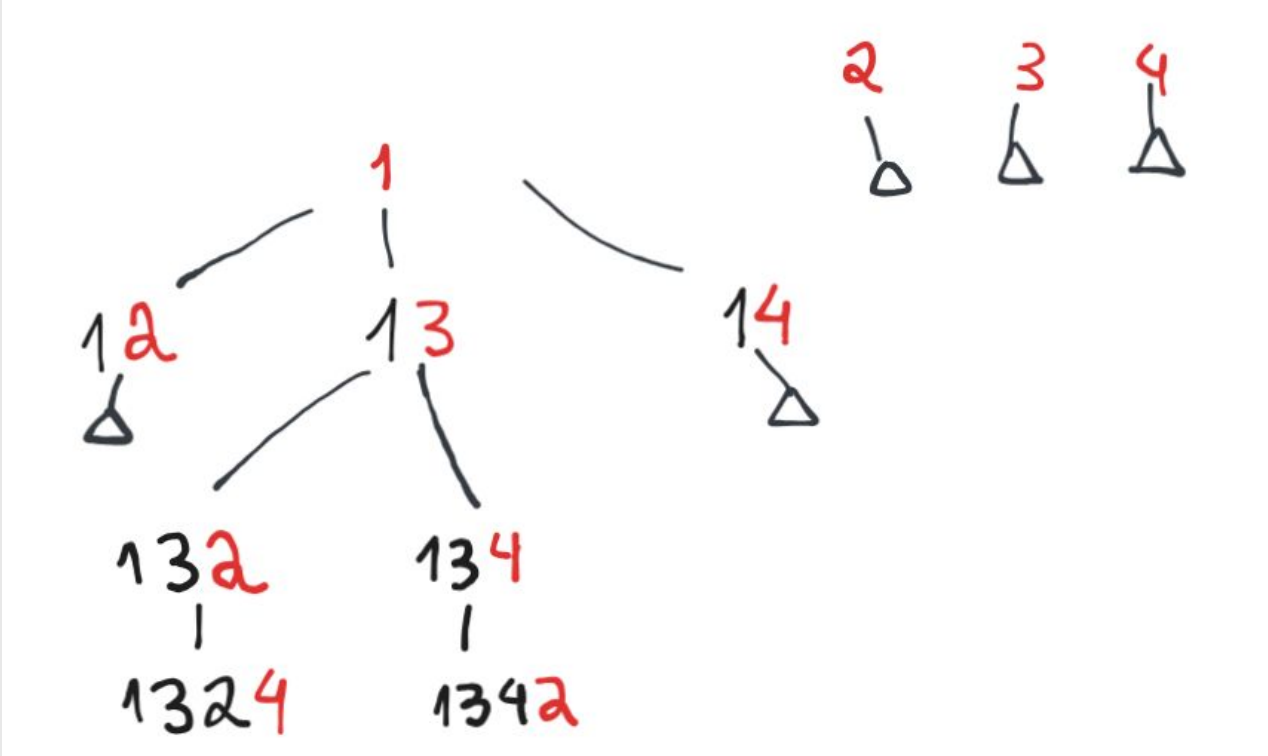
Universidade Federal de Pernambuco

Permutação como uma floresta



Floresta

Cada nó vai representando uma etapa na construção de uma permutação.



N = 4

Backtracking

O i não é importante para gerar as permutações, mas é importante para se obter o custo do TSP:

Cost(p) = Sum(C(p[i], p[i+1]))

index.cpp

```cpp
int best = INF;
int n = 4;
int C[n][n];
vector<int> perm;
int cur_cost = 0;
void backtracking_tsp(int i, vector<int>& used){
  int cnt = accumulate(begin(used),end(used),0);
  if(cnt == n){
    best = min(best, cur_cost);
    return;
  }
  for(int j = 0; j < n; j++) if(!used[j]){
    // faz
    used[j] = 1;
    perm.push_back(j);
    cur_cost += C[i][j];
    backtracking_tsp(j, used);
    // desfaz
    cur_cost -= C[i][j];
    perm.pop_back();
    used[j] = 0;
  }
}
```

Backtracking

Usando bitmasks

O i não é importante para gerar as permutações, mas é importante para se obter o custo do TSP:

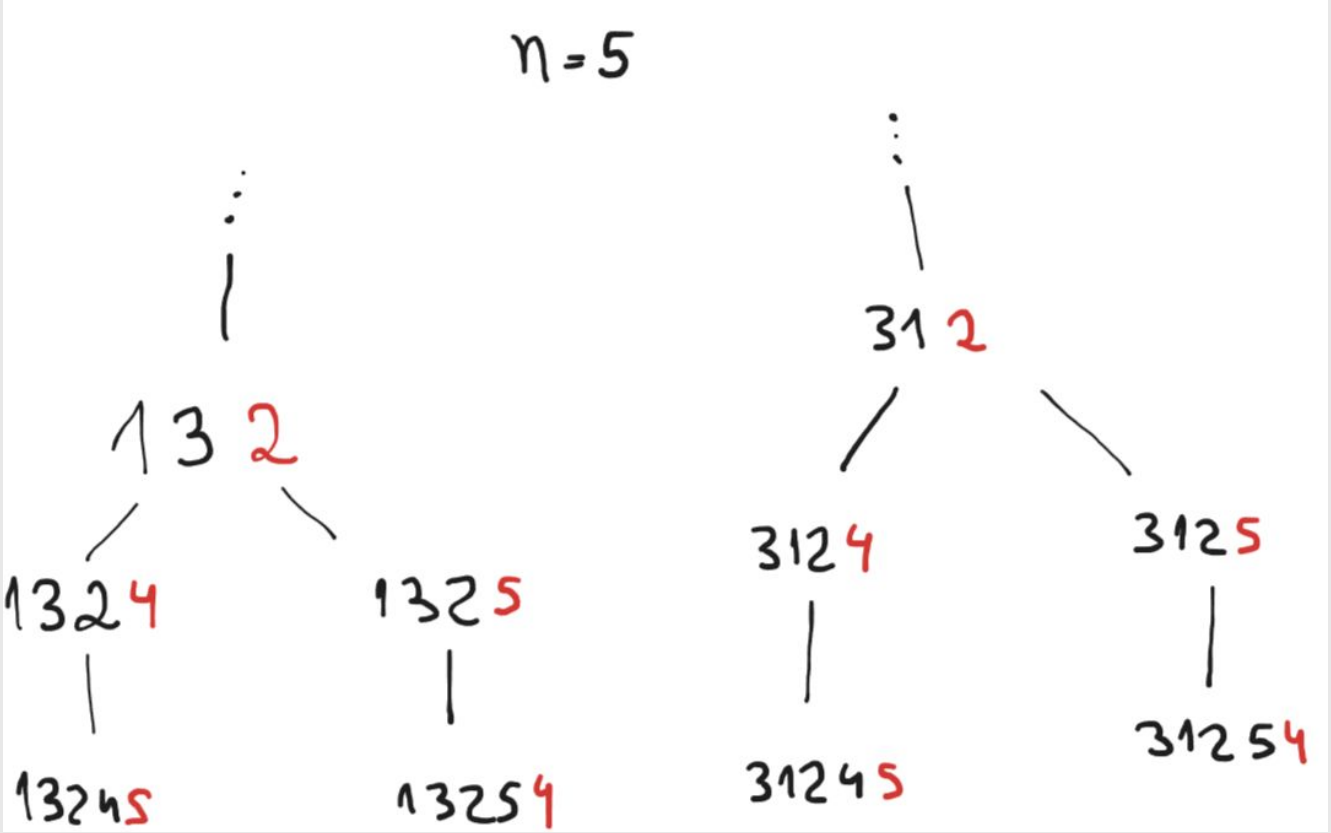
Cost(p) = Sum(C(p[i], p[i+1]))

index.cpp

```cpp
int best = INF;
int n = 4;
int C[n][n];
vector<int> perm;
int cur_cost = 0;
void backtracking_tsp(int i, int used){
  int cnt = __builtin_popcount(used);
  if(cnt == n){
    best = min(best, cur_cost);
    return;
  }
  for(int j = 0; j < n; j++) if(! (used & (1<<j))){
    // faz
    used ^= (1<<j);
    perm.push_back(j);
    cur_cost += C[i][j];
    backtracking_tsp(j, used);
    // desfaz
    cur_cost -= C[i][j];
    perm.pop_back();
    used ^= (1<<j);
  }
}
```

Floresta

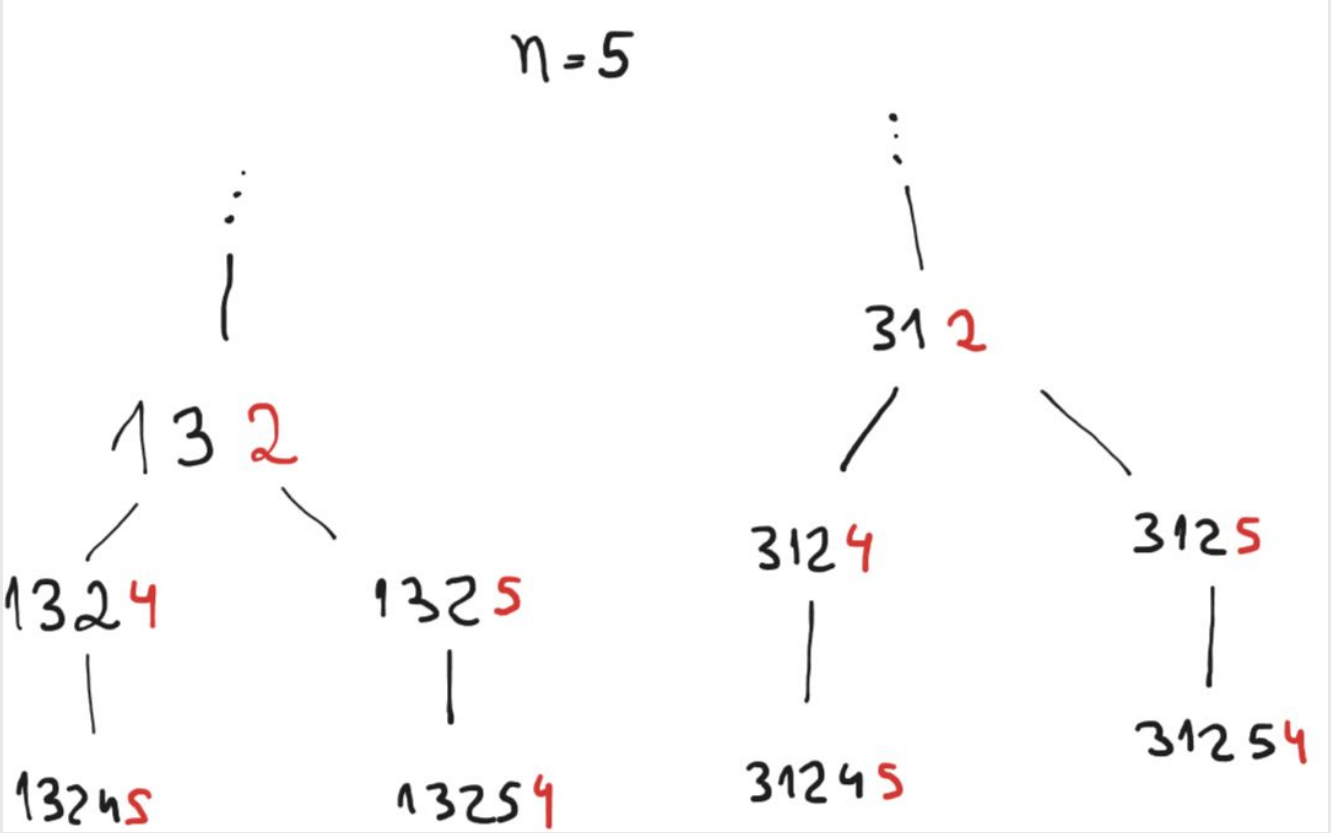
Qual semelhança entre esses dois ramos? (132 e 312)



Floresta

Qual semelhança entre esses dois ramos? (132 e 312)

Alcança estados e transições iguais na subárvore:



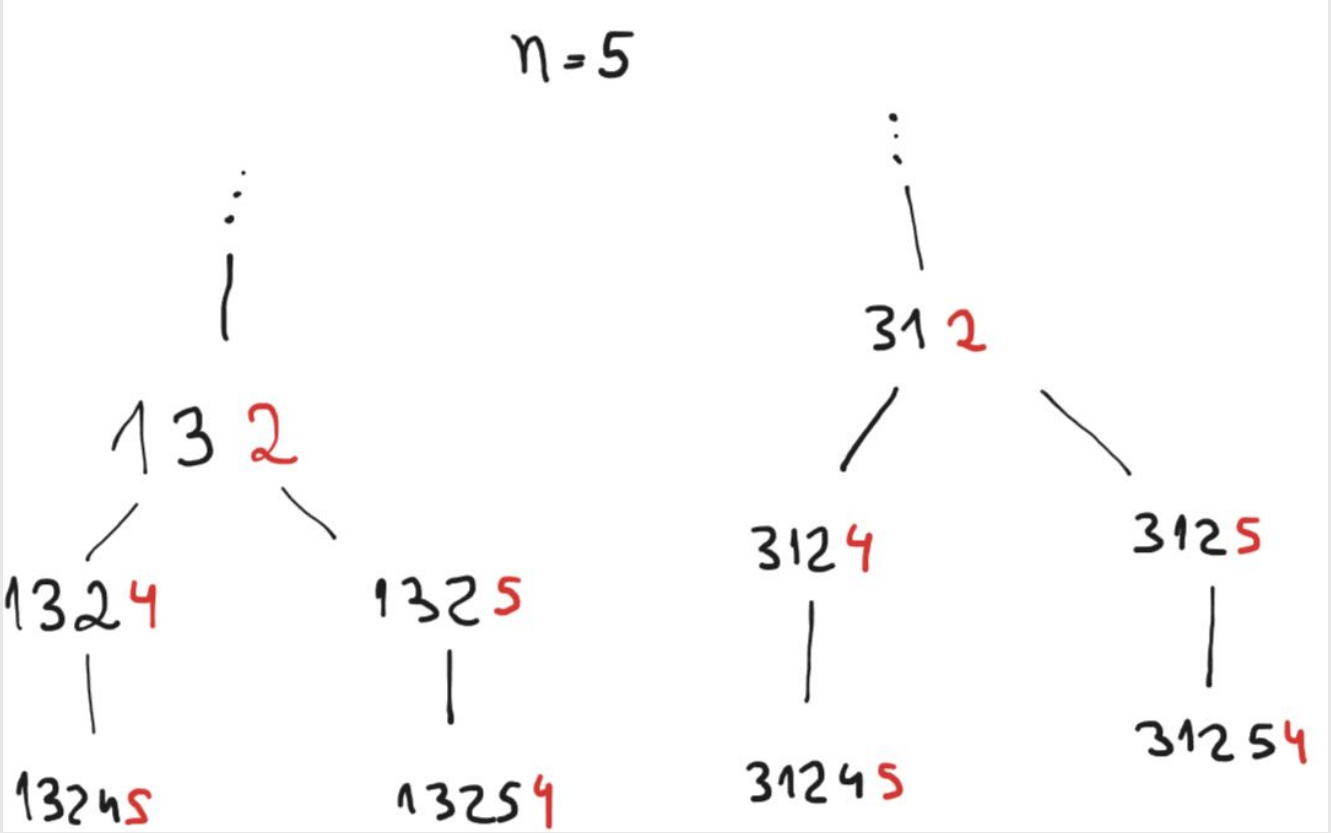
Floresta

Estados iguais: memoizar!

Vamos ter transições iguais quando o subset de números escolhidos for igual e o último número também.

Precisamos de uma bitmask e do último elemento escolhido:

dp[last][mask]



TSP recursivo

$O(n^2 . 2^n)$

index.cpp

```cpp
int n, m;
int g[20][20];
int dp[20][1<<20];

int solve(int i, int mask){
    if(mask == (1<<n) - 1){
        return g[i][0];
    }
    int& ans = dp[i][mask];
    if(ans != -1) return ans;
    ans = 2e9;
    for(int j = 0; j < n; j++){
        if(mask & (1<<j)) continue;
        ans = min(ans, g[i][j] + solve(j, mask ^ (1<<j)));
    }
    return ans;
}
```

TSP iterativo

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

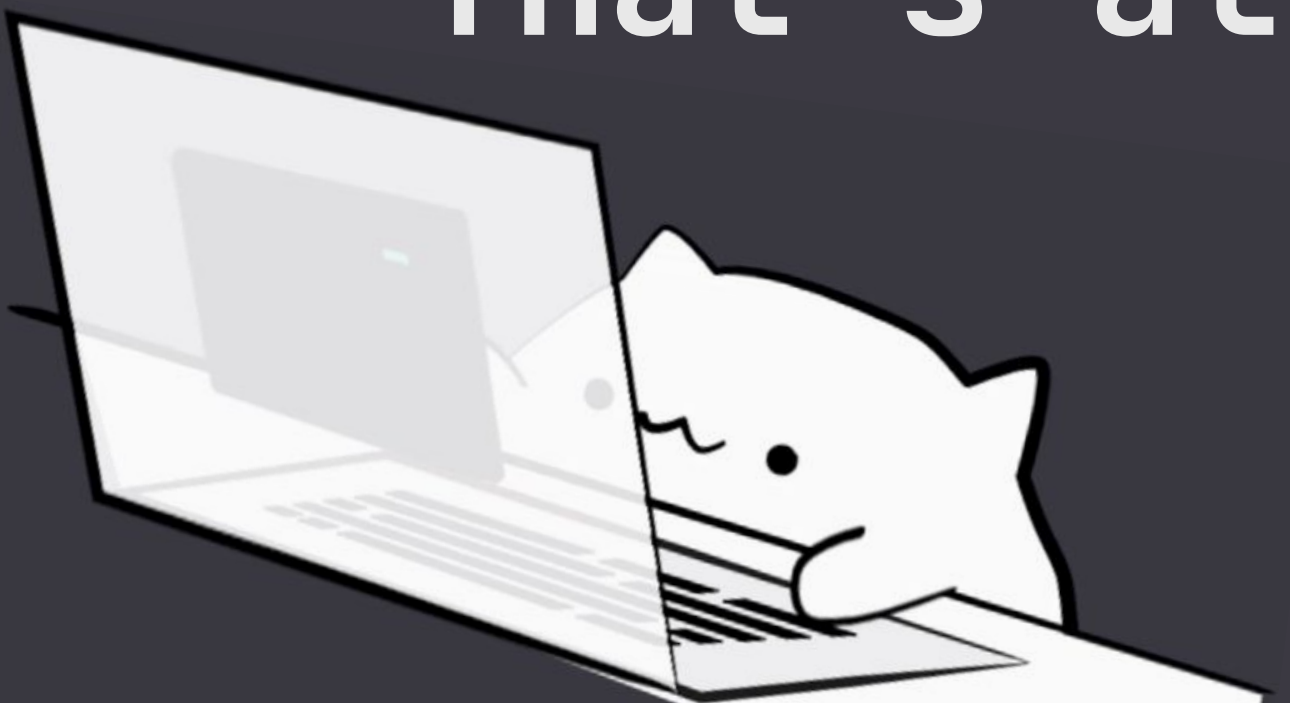
index.cpp

```cpp
const int inf = 2e9;
for(int mask = 0; mask < (1<<n); mask++)
    for(int i = 0; i < n; i++) dp[mask][i] = inf;

dp[1][0] = 0;
for(int mask = 1; mask < (1<<n); mask++){
    for(int i = 0; i < n; i++) if(mask & (1<<i)){
        for(int j = 0; j < n; j++){
            if(mask & (1<<j)) continue;
            dp[mask ^ (1<<j)][j] = min(dp[mask ^ (1<<j)][j], dp[mask][i] + g[i][j]);
        }
    }
}
int best = inf;
for(int i = 1; i < n; i++)
    best = min(best, dp[(1<<n) - 1][i] + g[i][0]);
```

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

That’s all, Folks!